



离散化与虚拟存储器实现

基于 UNIX V6++ V1 操作系统的内存管理实验

课程 操作系统课程设计

老师 邓蓉

姓名 王亦玮

学号 2351274

日期 2026 年 6 月 21 日

目录

1 实验概述	3
2 离散化内存管理	5
3 滚动条显示功能	9
4 Shell 功能增强	10
5 虚拟存储器——请求调页完整实现	11
6 总结	21

1 实验概述

1.1 背景与目标

本实验基于 UNIX V6++ V1 操作系统 (x86 32-bit 保护模式, 分页启用, 页目录基址 0x200000, 用户空间 8MB, 内核空间 4MB), 完成五项功能扩展。

离散化内存管理: 将物理页分配从 MapNode 链表替换为 BitMap 位图 (256 个 64 位整数覆盖 16384 页), 每次扫描 64 位, 时间复杂度 $O(N/64)$ 。新增 Page[] 引用计数数组 (以物理帧号为下标), 分配置 1、fork 父子各 +1、COW 打破旧 -1 新 1、归零才归还位图。

滚动条显示: 引入 200 行字符缓冲区 (m_Buffer[200][80]) , 将写入逻辑行 (m_WriteRow) 与屏幕视口起始行 (m_ViewStartRow) 解耦, 四快捷键在 TTyInput 中拦截。

Shell 增强: 行编辑从内核迁到用户态。内核仅三处改动 (扩展唤醒集、不回显控制字符、Canon 透传 Backspace)。Shell 端实现逐字符 read()、Tab 自动补全、Ctrl+W/S 历史导航。

系统调用 49/50/51: getppid、getpids、getproc, int 0x80 软中断, 用户态内联汇编宏。

虚拟存储器——请求调页完整实现: 缺页处理器扩展为四分支 (A-COW / B-栈 / C-数据按需 / D-文本哈希共享)。新增 DJB2 哈希表 ((inode, filePage) 键, 256 桶)、VMArea 链表 (TEXT/DATA/HEAP/STACK)、改进时钟置换算法。发现并修复 Fork 引用计数漏计 Bug。vmtest 五合一测试 5/5 PASS。

1.2 实验环境

项目	配置
宿主机	Windows 11 Home, Eclipse CDT Indigo SR2
编译器	MinGW i686-pc-mingw32-g++ 3.4.5
汇编器	NASM
仿真器	Bochs 2.6, 虚拟 32MB RAM, x86 i386
编译选项	-nostdlib -nostartfiles -nostdinc -fno-builtin -fno-rtti -fno-exceptions
用户空间	8MB (0x00000000-0x00800000), 2 个页表
内核空间	4MB (0xC0000000-0xC0400000), 1 个页表, 仅映射物理 0-4MB

2 离散化内存管理

2.1 原有机制与问题

原 PageManager 通过 MapNode 结构体数组（include/MapNode.h，最大 $0x200=512$ 项）记录空闲物理内存块。每个 MapNode 包含 m_Size（连续空闲块大小，页为单位）和 m_AddressIdx（起始页索引）。分配器 Allocator::Alloc() 采用首次适配算法：从 map[0] 开始线性扫描，找到第一个 m_Size 大于等于请求量的 MapNode，从该块分割出所需页数，更新 m_Size 和 m_AddressIdx。释放时 Allocator::Free() 将归还的块与前后相邻空闲块合并（coalescing）。

此设计有三个关键缺陷。第一，分配时间复杂度为 $O(n)$ ，其中 n 是 map[] 中有效（非零）MapNode 的数量。随着系统运行，内存碎片化导致 map[] 中分裂出大量小块， n 持续增长，分配越来越慢。第二，MapNode 不记录页级归属信息——它只记录“从哪开始、连续多少页空闲”，无法回答“物理页 0x405 被几个进程引用”这种问题，因此无法支持写时复制（COW）。第三，碎片本身加剧性能恶化，形成恶性循环。

2.2 BitMap 位图分配器设计

为解决上述问题，新增 BitMap 类（include/MapNode.h），用位图替代链表。核心成员：

- unsigned long long map[256]——256 个 64 位无符号整数，共 $256 \times 64 = 16384$ bit。每一位对应一个 4KB 物理页，0 表示空闲，1 表示已分配。可覆盖的最大地址空间为 $16384 \times 4096 = 64\text{MB}$ 。
- unsigned long m_AddressIdx——该位图管理的起始物理地址。对于内核页池，此值为 KERNEL_PAGE_POOL_START_ADDR (0x204000)；对于用户页池，此值为 USER_PAGE_POOL_START_ADDR (0x400000)。

- **int rows**——有效的 `uint64_t` 行数，由 `set()` 方法在初始化时根据管理页数计算：`rows = page_num / 64`。

初始化方法 `set(startAddr, page_num)` 设置 `m_AddressIdx` 和 `rows`，并将 `map[]` 前 `rows` 个元素全部清零（表示所有页初始空闲）。

位操作（`mm/Allocator.cpp`）分为两个基础函数。`is_free(bitmap, index)`：检查第 `index` 位是否为 0，返回 `!(bitmap.map[index/64] & (1ULL << (index%64)))`。`set_bit(bitmap, index, value)`：根据 `value` 将 `index` 位置 1 或清 0。这两个函数为上层 `Alloc/Free` 提供统一的位操作原语。

```
1 int BitMapAllocator::is_free(BitMap& bm, int idx) {
2     return !(bm.map[idx / 64] & (1ULL << (idx % 64)));
3 }
4 void BitMapAllocator::set_bit(BitMap& bm, int idx, int val) {
5     if (val) bm.map[idx/64] |= (1ULL << (idx%64));
6     else     bm.map[idx/64] &= ~(1ULL << (idx%64));
7 }
```

分配函数 `Alloc(bitmap, size)` 的流程：(1) 将 `size` 向上取整为需要的页数 `need = (size + 4095) / 4096`；(2) 从 `i=0` 到 `rows*64-1` 逐位扫描，调用 `is_free` 检查；(3) 若空闲则 `count++`，`count==1` 时记录 `start` 候选位置；若 `count==need` 则找到连续段；(4) 若遇已分配位则 `count` 重置为 0；(5) 找到后批量调 `set_bit` 将 `start` 到 `start+need-1` 位置 1；(6) 返回物理地址 `start*4096 + m_AddressIdx`。若扫完全部位仍未找到连续段，返回 0 表示分配失败。

释放函数 `Free(bitmap, size, addrIdx)` 的流程：(1) `start_page = (addrIdx - m_AddressIdx) / 4096`；(2) `pages = (size + 4095) / 4096`；(3) 循环调用 `set_bit` 将 `start_page` 到 `start_page+pages-1` 位清零。

时间复杂度分析：`Alloc` 扫描全部 `rows*64` 位，每次循环检查一个 64 位整数，总操作次数约为 `rows`，即 $O(N/64)$ （ N 为总页数）。`Free` 操作 k 页只需 k 次位清零， $O(k)$ 。分配和释放均不受碎片化程度影响，相比原 `MapNode` 链表的 $O(n)$ 性能提升显著。

内存布局：系统启动时，CMOS 读取物理内存总量（`PageManager::PHY_MEM_SIZE`）。内核页池起始地址 `KERNEL_PAGE_POOL_START_ADDR = 0x204000`（页目录和页表结构之后），大小 `KERNEL_PAGE_POOL_SIZE = 0x200000 - 0x4000`。用户页池起始地址 `USER_PAGE_POOL_START_ADDR = 0x400000`（4MB），大小 = `PHY_MEM_SIZE - 0x400000`。两个页池各有独立的 `BitMap` 实例，分别由 `KernelPageManager` 和 `UserPageManager` 管理（`kernel/Kernel.cpp` 全局对象

`g_KernelPageManager` 和 `g_UserPageManager`)。

2.3 Page[] 引用计数数组

在 `PageManager` 基类中新增 `int` 数组 `Page[USER_SPACE_SIZE / PAGE_SIZE]` (`include/PageManager.h`, 约 2048 个元素, 每个对应 4KB 页)。该数组以物理帧号 (`physical frame number = addr >> 12`) 为下标, 记录每个物理帧被多少个进程引用。

两个核心函数 (`mm/PageManager.cpp`):

```
1 unsigned long PageManager::AllocMemory(unsigned long size) {
2     unsigned long newaddr = m_pAllocator->Alloc(bitmap, size);
3     Page[newaddr >> 12] = 1;    // 新分配的帧, 引用计数初始为 1
4     Diagnose::Write("[BitMap] Alloc addr=0x%x\n", newaddr);
5     return newaddr;
6 }
7
8 unsigned long PageManager::FreeMemory(unsigned long size, unsigned
    long addr) {
9     Page[addr >> 12]--;        // 进程不再引用此帧
10    if (Page[addr >> 12] == 0) // 没有任何进程引用 → 归还位图
11        m_pAllocator->Free(bitmap, size, addr);
12    return 1;
13 }
```

引用计数在以下几个场景中变化:

- **分配时** (`AllocMemory`): `Page[frame]` 置为 1, 表示当前进程独占该帧。
- **Fork 时** (`NewProc` → `ModifyPageTable`): 父进程的可写页被标记为只读 (设置 COW), `Page[frame]` 递增 1 (父进程的引用已由 `AllocMemory` 计入, `ModifyPageTable` 为子进程新增 1); 子进程内联循环对每个 Present 页再递增 1。因此 fork 后 `Page[frame]` 总共 +2。
- **COW 打破时** (分支 A): 写入进程分配新帧 `newFrame` → `Page[newFrame]=1`; 旧帧引用减 1 → `Page[oldFrame]-`。
- **进程退出时** (`Exit` → `FreeMemory`): 遍历所有数据/栈 PTE, 对 Present 页逐页调 `FreeMemory`, `Page[frame]` 减 1。

核心语义: 物理帧只有在 `Page[frame]` 归零时才通过 `BitMapAllocator::Free` 归还位图。如果 `Page[frame] > 1` (被多个进程共享), 即使某个进程退出或打破 COW, 帧

也不会被回收——其他进程仍在使用它。这确保了 COW 场景下物理帧不会被错误释放。

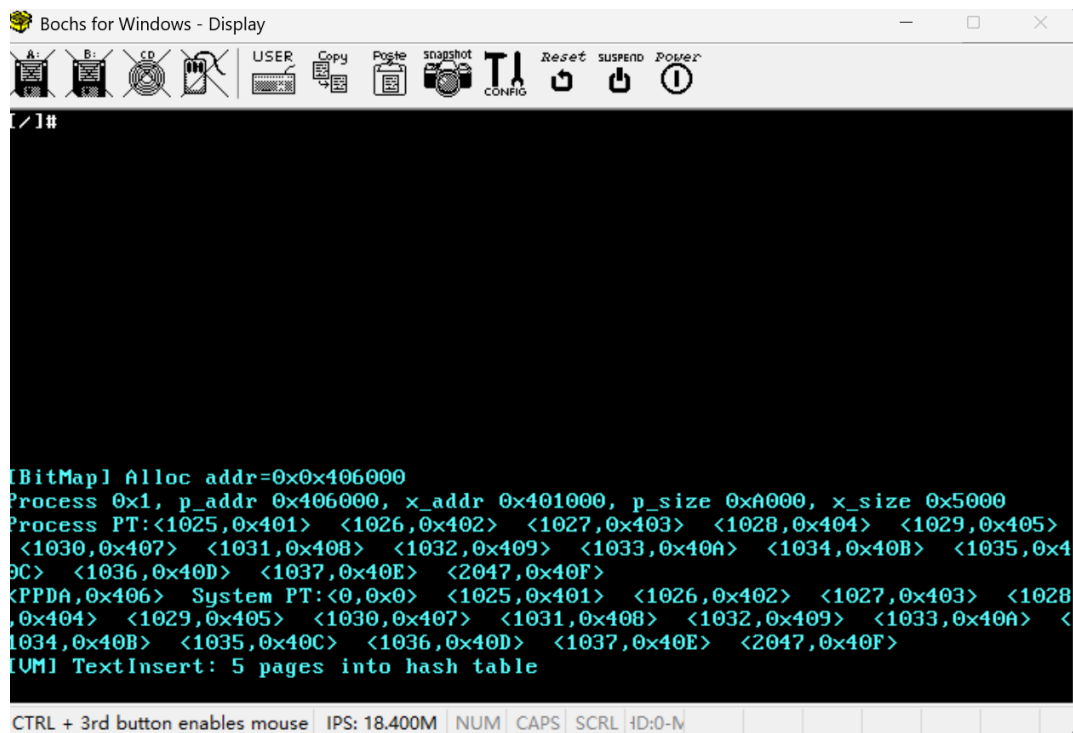


图 1: 系统启动时调试区显示的 BitMap 分配器日志。可见 [BitMap] Alloc addr=0x404000 等输出，证明位图分配器正常工作。

3 滚动条显示功能

3.1 原有机制与改进

CRT (tty/CRT.cpp) 和 Diagnose (kernel/Video.cpp) 直接写 VGA 显存 (0xB8000+0xC0000000)，超出屏幕即清屏。引入 200×80 字符缓冲 m_Buffer，两个关键变量将写入与显示分离：m_WriteRow（逻辑写入行，0-199，单调递增达上限时整体上移）和 m_ViewStartRow（屏幕第 0 行对应缓冲区的行号，正常自动跟随，按键时独立调整）。WriteChar 先写缓冲，仅当前行在视口内才同步显存。ScrollUp/Down 只改 ViewStartRow 后 Refresh (O(1200))。四快捷键 (Ctrl+E/D/R/F) 在 TTy::TTyInput() 字符入队前拦截。

4 Shell 功能增强

4.1 改动概要

内核侧 (tty/TTy.cpp) : (1) 扩展唤醒字符集——Tab(0x09)、Ctrl+W(0x17)、Ctrl+S(0x13)、Backspace(0x08)、Del(0x7F) 均触发 WakeUpAll; (2) 控制字符不回显; (3) Canon() 改为透传 Backspace (*pChar++ = ch)。

Shell 侧 (shell/main.c) : 重写 main1(), read(0,buf,512) 逐字符循环。Tab 调用 do_completion() 扫描 /bin 目录和内置命令列表, 收集匹配项, 计算最长公共前缀补全。Ctrl+W/S 在 history[20][512] 环形缓冲中导航。Backspace 删缓冲并 write(1,"\b\b",3)。

5 虚拟存储器——请求调页完整实现

5.1 改动文件清单

在原 BitMap + Page[] 基础上，新增 3 文件、重写 6 文件、配套 5 文件，共 14 文件。

类别	文件与改动
新增 (3)	HashTable.h/cpp——共享页哈希表, (inode,filePage) 键, DJB2 哈希, 256 桶。VMArea.h——虚存区域描述符。
重写 (6)	Exception.cpp (第 135–293 行)——PageFault 四分支, 三段式 COW 搬运。 ProcessManager.cpp——Exec 传 inode+ 插哈希表 + 切惰性 (第 479–738 行); NewProc 保存/恢复 VMArea+ 子进程全 Present 计数 (第 83–185 行)。 Process.cpp——SStack 直写硬 PTE (第 352–390 行); SBreak 管理 HEAP VMArea (第 416–465 行); Exit 清理哈希表 (第 181–302 行)。 MemoryDescriptor.cpp——vm_area 增删查; EstablishUserPageTable 文本先可用后惰性。 Kernel.h/cpp——全局 HashTable。Utility.h/cpp——DJB2 哈希。
配套 (5)	MemoryDescriptor.h (VMArea 链表成员 + Inode 参数); mm/Makefile (hashtable.o) ; boot/boot.s (KERNEL_SIZE 180→200) ; program/vmtest.c + Makefile (五合一测试)。

5.2 核心数据结构

共享页哈希表 (HashTable)。根据设计文档要求，以 (inode, filePage) 为键管理进程间共享的物理页框。我实现了独立的 HashTable 类 (mm/HashTable.cpp, 102 行)，不依赖内核动态内存分配。

HashEntry 结构体包含五个字段：**h_inode** (Inode*, 可执行文件的 inode 指针, 作为键的第一部分)、**h_filePage** (unsigned int, 文件内页号 = offset / 4096, 作为键的第二部分)、**h_frame** (unsigned int, 共享的物理帧号, 命中时直接填入 PTE)、**h_refCount** (unsigned int, 当前有多少进程共享此帧, Insert 时初始为 1, AddRef 递增, Release 递减, 归零时从链表删除)、**h_next** (HashEntry*, 冲突链指针)。

桶数量 HASH_SIZE = 256, 桶数组为 m_Buckets[256] (HashEntry* 类型, 初始全 NULL)。 **条目池** m_EntryPool[512] (HashEntry 结构体数组, HASH_SIZE × 2), 使用 m_PoolIdx (int, 初始 0) 顺序分配, Insert 时 PoolIdx++。 512 个条目对应最多 512 个不同的 (inode, filePage) 组合, 对于教学实验规模绰绰有余。

哈希函数 (kernel/Utility.cpp 第 224-238 行) 采用 **DJB2 变体**: 以 5381 为初值, 逐字节混入两个 unsigned long 参数的各字节, 共 8 轮 (hash = (hash << 5) + hash + byte), 返回 32 位哈希值。 HashTable 内部将其对 HASH_SIZE 取模得到桶索引。

四种操作: (1) **Lookup**(inode, filePage)——计算桶索引, 遍历该桶的冲突链, 比较 h_inode == inode 且 h_filePage == filePage, 匹配则返回 HashEntry*, 遍历完返回 NULL。 (2) **Insert**(inode, filePage, frame)——若 PoolIdx ≥ 512 返回 NULL; 从池中取 &m_EntryPool[PoolIdx++], 填充各字段, h_refCount=1, h_next = m_Buckets[bucket], m_Buckets[bucket] = entry (头插法)。 (3) **Release**(inode, filePage)——同桶查找, 命中后 h_refCount--, 若归零则从链表摘除 (修改前驱的 next 指针), 返回 h_frame 供调用者释放物理帧。 (4) **AddRef**(inode, filePage)——同桶查找, 命中后 h_refCount++。

Exec 阶段: 文本段的所有物理帧逐一调 **ht.Insert**(pInode, tp, textFrame + tp) 注册进哈希表。 **缺页阶段** (分支 D): 调 **ht.Lookup**(vma->vm_inode, filePage) 仅需一次哈希计算加一次链表遍历 (平均链长 = 条目数 / 256, 极短)。 **Exit 阶段**: 遍历进程的文本 VMArea, 逐页调 **ht.Release** 递减引用。 当多个进程执行同一可执行文件时, 文本页通过哈希表共享同一物理帧——**全程无磁盘 I/O**。

VMArea 链表。 设计文档要求在缺页时遍历进程的 **vm_list** 确定 cr2 落在哪个 **vm_area**。 我定义了 **VMArea 结构体** (include/VMArea.h) 和枚举 **VMAType** { VMA_TEXT=0, VMA_DATA=1, VMA_HEAP=2, VMA_STACK=3 }。 每个 VMArea 包含: **vm_start**、**vm_end** (unsigned long, 起止虚拟地址, vm_end 为 exclusive); **vm_type** (VMAType); **vm_foff** (unsigned long, 文件内偏移, 仅 VMA_TEXT 使用); **vm_inode** (Inode*, 仅 VMA_TEXT 使用, 指向可执行文件的 inode, 供分支 D 获取哈希表键); **vm_next** (VMArea*, 链表指针)。

MemoryDescriptor 中新增三个成员 (include/MemoryDescriptor.h): **m_VMList**

(VMArea*, 链表头指针)、**m_VMAreas[8]** (VMArea 内嵌数组, 避免内核动态分配)、**m_VMCount** (int, 当前已使用数量, 上限 8)。三个方法 (MemoryDescriptor.cpp 第 49–84 行):

FindVMArea(unsigned long vaddr): 从 m_VMList 开始遍历, 对每个 VMArea* vma, 若 $vaddr \geq vma->vm_start$ 且 $vaddr < vma->vm_end$, 返回 vma; 遍历完返回 NULL。调用者据此判断缺页地址所属区域类型。

AddVMArea(start, end, type, foff, inode): 若 $m_VMCount \geq 8$ 则忽略。从 **m_VMAreas[m_VMCount++]** 取空闲元素, 填充各字段, $vm_next = m_VMList$, $m_VMList = vma$ (头插法, $O(1)$)。

RemoveAllVMAreas(): $m_VMList = NULL$, $m_VMCount = 0$ 。在 EstablishUserPageTable 开头调用, 清除旧 exec 残留的区域描述符。

Exec 时 **EstablishUserPageTable** 创建 TEXT (start=parser.TextAddress, end 对齐到页边界)、DATA (类似)、STACK (start=USER_SPACE_SIZE-StackSize, end=USER_SPACE_SIZE) 三个区域。**SBreak** 在堆扩展时: 若 oldEnd 处已有 HEAP VMArea 则扩展其 vm_end , 否则新增 VMA_HEAP。**Fork** 时子进程通过 PPDA 拷贝 (CopySeg) 自然继承父进程的全部 VMArea 链表。**Exit** 时随 MemoryDescriptor 释放。

双页表架构。软件页表 ($m_UserPageTableArray$, PageTable*) 由 MemoryDescriptor::Initialize() 从内核页池分配 2 页 ($8KB = 2 \text{ 页表} \times 1024 \text{ 条目} \times 4 \text{ 字节 PTE}$)。**PageTableEntry** 为位域结构体 (include/PageTable.h): **Present** (1 bit)、**Read-Writer** (1 bit)、**UserSupervisor** (1 bit)、**Accessed** (1 bit, CPU 硬件自动置位)、**Dirty** (1 bit, CPU 硬件自动置位)、**ForSystemUser** (3 bit, bit0= 已换出标记, bit1= 文本惰性标记)、**PageBaseAddress** (20 bit, 存储物理帧号 / 文件页号 / swap 块号, 取决于 ForSystemUser 和 Present 的状态)。

硬件页表 (Machine::GetUserPageTableArray()) 位于物理地址 0x202000 和 0x203000, 内核虚拟地址 0xC0202000 和 0xC0203000。这两个物理页由 **InitUserPageTable** (machine/Machine.cpp) 在系统启动时分配并映射。**MapToPageTable**() (MemoryDescriptor.cpp 第 150–175 行) 将所有软件 PTE 逐条拷贝到硬件 PTE, 并在末尾强制设置第 0 页 (runtime 入口) 为 Present=1、ReadWriter=1、PageBaseAddress=0。然后调 **FlushPageDirectory**() (mov cr3, 0x200000) 刷新全部 TLB。

关键设计决策: 缺页处理的分支 C/D 和 SStack 直写指定硬件 PTE 条目, 不经过 MapToPageTable 的全量拷贝 + FlushPageDirectory 路径。原因: 若 MapToPageTable 被调用, FlushPageDirectory 清空所有 TLB 条目——包括惰性文本页 (Present=0)

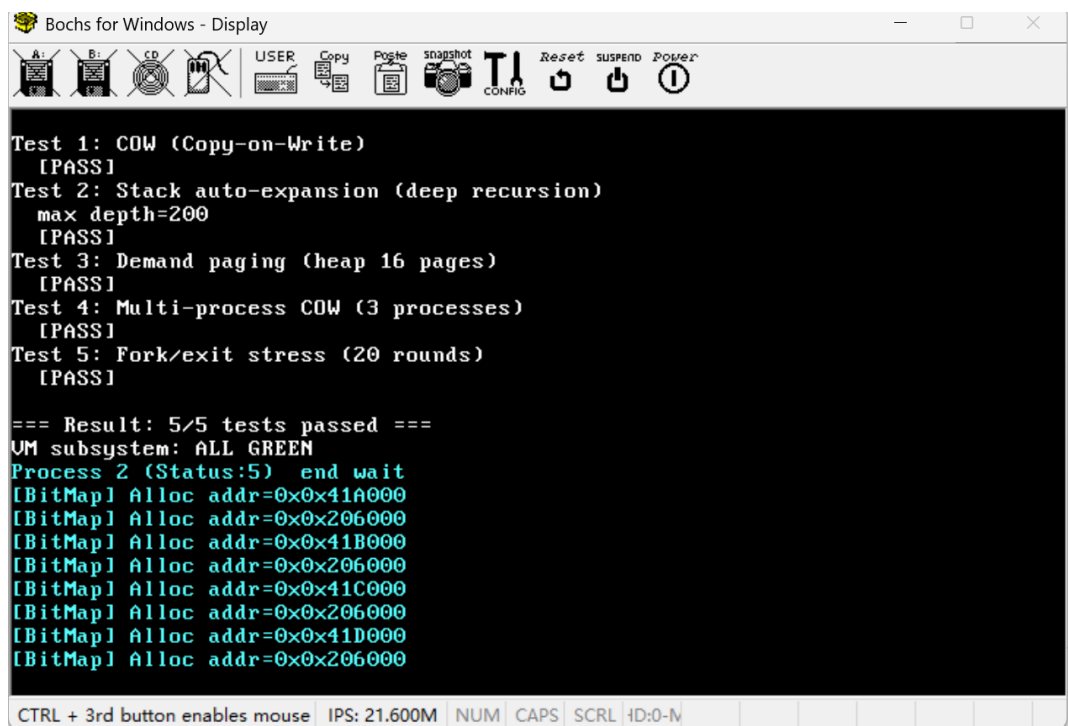
此前被缓存的 Present=1 条目。TLB 清空后，下一次指令预取不得不走页表遍历，发现 Present=0，再次触发缺页（分支 D）——形成「全局刷新 → 连锁缺页」的级联反应。直写指定 PTE 仅伴随单页 invlpg，完全规避这一问题。

5.3 测试程序 vmtest

vmtest.c（program/vmtest.c，208 行）的 main1() 依次运行 5 项测试：

1. **Test 1——COW**。cow_buf[8192] 页对齐，填'A'。fork，子改前半为'B' 自证隔离，exit 返回错误码。父 wait 后验仍全'A'。覆盖分支 A。
2. **Test 2——栈扩展**。200 层递归 × 256B 局部变量 ≈ 50KB 栈。每层以 n&0xFF 填 frame，递归后逐字节验证未被破坏。覆盖分支 B。
3. **Test 3——按需调页**。sbrk(16×4096) 不给页。逐页写标记字节并回读——每页首次写触发分支 C 零填充。覆盖分支 C。
4. **Test 4——多进程 COW**。父 X → 子改前 1/3 为 Y → 孙改中 1/3 为 Z。三进程各验隔离。覆盖分支 A 多级。
5. **Test 5——Fork 压力**。20 轮 fork+ 子 exit+ 父 wait。引用归零则帧误释放 → Invalid MM access。覆盖 Page[] 计数。

预期输出：=== Result: 5/5 tests passed === VM subsystem: ALL GREEN。



The screenshot shows a Bochs for Windows - Display window. The title bar is "Bochs for Windows - Display". The window contains a black terminal area with white text. The text shows the results of five tests: Test 1: COW (Copy-on-Write) [PASS], Test 2: Stack auto-expansion (deep recursion) max depth=200 [PASS], Test 3: Demand paging (heap 16 pages) [PASS], Test 4: Multi-process COW (3 processes) [PASS], and Test 5: Fork/exit stress (20 rounds) [PASS]. Below the tests, it says "=== Result: 5/5 tests passed ===" and "VM subsystem: ALL GREEN". Then it shows "Process 2 (Status:5) end wait" followed by several lines of memory allocation logs: "[BitMap] Alloc addr=0x0x41A000", "[BitMap] Alloc addr=0x0x206000", "[BitMap] Alloc addr=0x0x41B000", "[BitMap] Alloc addr=0x0x206000", "[BitMap] Alloc addr=0x0x41C000", "[BitMap] Alloc addr=0x0x206000", "[BitMap] Alloc addr=0x0x41D000", and "[BitMap] Alloc addr=0x0x206000". At the bottom of the window, there is a status bar with text: "CTRL + 3rd button enables mouse | IPS: 21.600M | NUM | CAPS | SCRL | ID:0-N".

图 2: vmtest 五合一测试程序运行结果。5 项测试全部 [PASS], 调试区同步出现 [VM] ZeroFill、[VM] TextShare 等诊断日志。

5.4 exec 系统调用

Exec() (ProcessManager.cpp 第 479–738 行) 按以下七步执行:

步骤 1——解析 PE。调 fileMgr.NameI(NextChar, OPEN) 按路径名查找可执行文件的 inode。调 parser.HeaderLoad(pInode) 读取 PE 可选头, 获取 TextAddress、TextSize、DataAddress、DataSize、StackSize。

步骤 2——预分配物理帧。文本段调 userPgMgr.AllocMemory(pText->x_size) 分配连续物理内存, 地址存入 pText->x_caddr。PPDA+ 数据 + 栈作为整体调 Expand(USIZE + DataSize + StackSize) 扩展 p_size 并分配连续物理内存, 地址存入 p_addr。PPDA 占第 0 页, 数据从 p_addr + 4096 开始, 栈紧随其后。

步骤 3——构建 fakeStack。在内核页池分配临时缓冲区, 从高地址向低地址逐一压入: argv 各字符串 → 对齐 16 字节 → 压 0 → 压 argv 指针数组 → 压 argc。同步记录用户栈 esp。

步骤 4——建立页表。调 EstablishUserPageTable(parser.TextAddress, ..., pInode) (MemoryDescriptor.cpp 第 84–109 行): ClearUserPageTable() 清零全部 2048 个 PTE 并设 UserSupervisor=1; 为 text/data/stack 分别创建 VMArea 加入 vm_area

链表；文本段以 `x_caddr` 为帧号调 **MapEntry** 设为 `Present=1`、`ReadWriter=0`（只读，供 `Relocate` 内核态写入）；数据段以 `p_addr/4096+1` 为起始帧号调 **MapEntry** 设为 `Present=1`、`ReadWriter=1`；栈段类似。

步骤 5——载入 PE 数据。调 `parser.Relocate(pInode, sharedText)` 将可执行文件的 `.text`、`.data`、`.rdata`、`.bss` 各 section 从 inode 的数据块读入预分配的物理帧。此时文本页为 `Present`，内核态写入不会触发缺页。

步骤 6——文本惰性化。若 `sharedText == 0`（本进程是该可执行文件的首个加载者），遍历所有文本页：对第 `tp` 页调 `ht.Insert(pInode, tp, textFrame+tp)` 将物理帧注册进哈希表。然后逐页修改软件 PTE 和硬件 PTE：**Present** 设为 `0`，**PageBaseAddress** 改为文件内页号 `i`（而非物理帧号），**ForSystemUser** 设为 `2`（标记文本惰性）。最后 `invlpg` 刷新第一个文本页的 TLB。

步骤 7——拷贝参数并返回。调 `Utility::MemCopy` 将 `fakeStack` 中备份的命令行参数拷贝到用户栈虚拟地址。释放 `fakeStack` 临时缓冲区。设置返回上下文：`pContext->eip = 0`（runtime 入口）、`pContext->xcs = USER_CODE_SEGMENT_SELECTOR`、`pContext->eflags = 0x200`、`pContext->esp = esp`（用户栈顶）、`pContext->xss = USER_DATA_SEGMENT_SELECTOR`。

5.5 缺页异常处理器

PageFault 函数（`Exception.cpp` 第 135–293 行）实现统一的四分支入口。

入口逻辑（第 142–153 行）：读 **CR2** 寄存器得缺页线性地址；若 `context->xcs` 非 `USER_MODE` 则 **Panic**（内核态缺页不可恢复）；`vPageIdx = cr2 >> 12`；分别从软件 PTE 和硬件 PTE 取对应条目；调 `FindVMArea(cr2)` 遍历 `vm_area` 链表定位地址所属区域。

分支 A——COW 写时复制（第 156–202 行）。判定条件：**error_code bit1=1**（写故障）且软件 PTE **Present=1** 但 **ReadWriter=0** 且 `cr2` 在数据段或栈区范围且 `Page[PageBaseAddress] ≥ 1`。

独占情形（`Page[oldFrame] == 1`）：直接设 `ReadWriter=1`，无需拷贝——说明其他共享者早已打破 COW 或退出，只剩当前进程还在引用此帧。

共享情形（`Page[oldFrame] > 1`）——三段式搬运：

```
1 unsigned int oldFrame = entrys[vPageIdx].m_PageBaseAddress;
2 // 共享页：分配新帧 + 拷贝 + 旧引用减 1
```



```

3 unsigned long newAddr = userPageMgr.AllocMemory(PAGE_SIZE);
4 if (newAddr == 0) { userPageMgr.EvictOnePage(); newAddr = ...; }
5 if (newAddr == 0) goto sigsegv;
6
7 // 三段式搬运：内核仅映射 0--4MB，用户帧 >= 4MB 不可直访
8 KernelPageManager& kpm = Kernel::Instance().GetKernelPageManager();
9 unsigned long tmpPage = kpm.AllocMemory(PAGE_SIZE);          // 临时页 <
                        4MB
10 if (tmpPage == 0) { userPageMgr.FreeMemory(PAGE_SIZE, newAddr); goto
    sigsegv; }
11
12 unsigned long userVA = cr2 & 0xFFFFF000;
13 DWordCopy((int*)userVA, (int*)(tmpPage | 0xC0000000), 1024); // 用
    户 VA → 临时页
14 hwEntrys[idx].m_PageBaseAddress = newAddr >> 12; invlpg(userVA); //
    PTE 切新帧
15 DWordCopy((int*)(tmpPage | 0xC0000000), (int*)userVA, 1024); // 临
    时页 → 用户 VA
16 kpm.FreeMemory(PAGE_SIZE, tmpPage);
17 Page[oldFrame]--; Page[newAddr >> 12] = 1;

```

为什么用三段式。内核页表仅映射物理 0-4MB (InitPageDirectory 只设 PDE[768] 映射 0xC0000000-0xC0400000)，用户物理帧从 0x400000 (4MB) 起。若用 physAddr | 0xC0000000 计算内核虚拟地址，所得 $\geq 0xC0400000 \rightarrow$ 对应 PDE 第 769 项，未被 InitPageDirectory 或 InitUserPageTable 初始化 $\rightarrow$ Present=0 $\rightarrow$ 内核态缺页 $\rightarrow$ Double Fault $\rightarrow$ Panic。用户 VA 走用户 PDE (第 0/1 项已由 InitUserPageTable 初始化且 UserSupervisor=1 允许内核访问)，全程不碰 physAddr | 0xC0000000。

分支 B——栈自动扩展 (第 204-212 行)。三重判定：(1) $cr2 < USER_SPACE_SIZE - m_StackSize$ (缺页地址低于当前栈底)；(2) $cr2 \geq context->esp - 8$ (地址靠近用户态栈指针，排除野指针)；(3) $DataSize + StackSize + PAGE_SIZE < USER_SPACE_SIZE - DataStartAddress$ (尚有剩余空间)。SStack (Process.cpp 第 352-390 行)： $m_StackSize += PAGE_SIZE \rightarrow$ AllocMemory 分配一页 $\rightarrow$ 写软件 PTE 对应条目 $\rightarrow$ 调 FindVMArea 找栈 VMArea 并扩展其 vm_start $\rightarrow$ 直写硬件 PTE (不调 MapToPageTable) $\rightarrow$ 单页 invlpg $\rightarrow$ 零填充。关键决策：不调 MapToPageTable——避免 FlushPageDirectory 全刷 TLB 引发惰性文本页连锁缺页。

分支 C——数据/堆按需调页 (第 214-249 行)。判定：Present=0 且 vma->vm_type 为 VMA_DATA 或 VMA_HEAP。AllocMemory 分配帧 (位图满则 EvictOnePage

换出)。若 ForSystemUser bit0=1 (此页曾被换出) → 从 PageBaseAddress 取 swap 块号 → **BufferManager::Swap**(swapBlock, frame, B_READ) 换入 → **FreeSwap** 归还块。否则 (首次访问) → 经用户 VA **零填充**。最后写软件 PTE (Present=1、ReadWriter=1、PageBaseAddress=frame>>12、ForSystemUser=0)、硬件 PTE、Page[frame>>12]=1、invlpg。**惰性本质**: sbrk → SBreak 只改 m_DataSize + 注册 **VMA_HEAP**, 不给物理页, 首次写入由分支 C 按需分配。

分支 D——文本段哈希共享 (第 251–279 行)。**判定**: Present=0 且 ForSystemUser bit1=1 且 vma->vm_type==VMA_TEXT 且 vm_inode ≠ NULL。从 PTE 取 filePage (Exec 阶段写入的文件内页号), 从 VMArea 取 inode → **ht.Lookup**。命中 → 软件 PTE 设为 Present=1、ReadWriter=0、PageBaseAddress=he->h_frame、ForSystemUser=0, 拷贝到硬件 PTE, **AddRef** 递增引用计数, invlpg。未命中 → 打印 **[VM] TextMiss** 诊断后 goto sigsegv (理论上不应发生, Exec 已 Insert)。

sigsegv (第 281–286 行): **PSignal**(SIGSEGV) → 若 IsSig() 则 **PSig** 终止进程, 诊断输出 Invalid MM access va=0x....

5.6 Fork 引用计数 Bug 的发现与修复

在 **vmtest Test 5** (20 轮 fork/exit) 的调试过程中, 第 5 轮左右父进程收到 SIGSEGV——**Invalid MM access**。

根因分析。**ModifyPageTable** (ProcessManager.cpp 第 13–32 行) 原代码条件为 if (entrys[i].m_Present && entrys[i].m_ReadWriter) { ... Page[frame]++; }。第 1 轮 fork 时父进程的可写页 (ReadWriter=1) 被正确标为只读 (ReadWriter=0) 并 Page[]++。但子进程的页表拷贝自父进程——父进程的页已标为只读, 所以子进程页表的 ReadWriter 也是 0——**ModifyPageTable(子)** 的条件 ReadWriter==1 为假, **跳过了子进程所有数据页** → 子进程的引用未被计数。第 1 轮子进程 Exit 时 FreeMemory 将 Page[F] 减到比预期更低的值。第 2 轮 fork 时 **ModifyPageTable(父子)** 由于 ReadWriter 已为 0 **全部跳过**。第 2 轮子进程 Exit 时 FreeMemory 将 Page[F] 减到 0 → **物理帧被释放回位图!** 但父进程的 PTE 仍指向该帧 (Present=1)。父进程后续访问该帧时——该帧可能已被位图重新分配给其他用途——触发缺页且不匹配任何分支 → sigsegv → Invalid MM access → **进程终止**。

修复方案 (ProcessManager.cpp 第 120–141 行)。父进程仍走 **ModifyPageTable** (仅对可写页计数的原逻辑)。子进程不再调 **ModifyPageTable**, 改为**内联循环**: 遍历数据段范围 (dataStart → dataEnd) 和栈范围 (stkStart → stkEnd), 对每个 if (childEntrys[i].m_Present)——**无论 ReadWriter 是 0 还是 1——一律置**

ReadWriter=0、Page[PageBaseAddress]++。循环次数最多 2048 次（2 页表 × 1024 条目），对 fork 的延迟影响可忽略。修复后 Test 5 通过——20 轮 fork/exit 后引用计数不泄露，父进程可继续正常访问所有数据页。

5.7 页面置换——时钟算法

当 AllocMemory 扫描位图找不到连续空闲页时，触发 **EvictOnePage** (mm/PageManager.cpp 第 77-127 行)。

时钟算法流程：维护静态变量 **hand**（初始为 0，记录上次扫描停止位置，保证公平轮转）。**第一轮**——遍历硬件页表，对每个可写且存在的页将 **Accessed** 位清零。CPU 硬件在每次页访问时自动将对应 PTE 的 Accessed 位置 1，此特性被时钟算法利用：第一轮清空所有「时钟」，第二轮检查哪些页的时钟未被重新拨动。**第二轮**——从 hand 位置开始循环扫描（模 USER_SPACE_SIZE/PAGE_SIZE），跳过索引 0（runtime 入口页）、不存在的页、只读页、Accessed 仍为 1 的页。找到第一个 **Accessed==0** 的可写页作为受害者。

检查 Dirty 位：若为 1（脏页，进程写过），调 **AllocSwap** 分配 8 个连续磁盘块（=4096 字节），调 **BufferManager::Swap**(swapBlock, physFrame<<12, PAGE_SIZE, B_WRITE) 将脏数据写盘。更新软件 PTE：PageBaseAddress 存 swap 块号，ForSystemUser 的 bit0 置 1（标记「已换出」）。若 Dirty 为 0（干净页，与磁盘副本一致），直接跳过写盘。最后——软件 PTE 的 Present 清 0；硬件 PTE 的 Present 清 0；**invalpg** 刷新该地址的 TLB；调 **FreeMemory**(PAGE_SIZE, physFrame<<12) 归还物理帧；hand = (i+1) % Npages 前进时钟指针。

后续换入：当进程访问曾被换出的页时，缺页进入分支 C。分支 C 检测 ForSystemUser bit0=1 → 从 PageBaseAddress 取 swap 块号 → **Swap**(swapBlock, frame, B_READ) 从磁盘读回 → **FreeSwap** 归还磁盘块 → 建 PTE 映射。Swap 内部通过 BufferManager 的 DMA 通道走磁盘 I/O，数据直接从磁盘控制器写入物理帧，不经过内核别名地址——因此不受「内核仅映射 0-4MB」的限制。

5.8 进程退出时的清理

Exit()（Process.cpp 第 181-302 行）在写入 u 结构到交换区之后、释放内存描述符之前，做三阶段的物理资源回收：

第一阶段——释放数据段和栈区。遍历软件 PTE 的数据段范围和栈区范围（分别

由 `m_DataStartAddress`、`m_DataSize`、`m_StackSize` 计算起止页号)。对每个条目：若 `Present=1`，调 `FreeMemory(PAGE_SIZE, PageBaseAddress<<12)`——这会将 `Page[]` 减 1，仅在归零时归还位图；若 `Present=0` 且 `ForSystemUser bit0=1`（曾换出），调 `FreeSwap(PAGE_SIZE, PageBaseAddress)` 归还交换区块。

第二阶段——清理哈希表。遍历进程的 `vm_area` 链表。对每个类型为 `VMA_TEXT` 且 `vm_inode` 非空的区域，计算文本页数 $((vm_end - vm_start) + PAGE_SIZE - 1) / PAGE_SIZE$ ，对每页调 `ht.Release(vma->vm_inode, tp)`。`Release` 递减 `h_refCount`；归零时从桶链表删除条目——但条目的物理帧不在此释放（物理帧属于 `Text` 结构体的 `x_caddr` 整块分配，由 `p_textp->XFree()` 统一回收）。

第三阶段——释放内核资源。调 `u.u_MemoryDescriptor.Release()`——将 `m_UserPageTableArray` 指针转为物理地址（减去 `0xC0000000`），调 `KernelPageManager::FreeMemory` 归还 2 页（8KB）的软件页表内存。随后调 `FreeMemory(PAGE_SIZE, current->p_addr)` 释放 PPDA 物理帧。

5.9 对标表

设计要求	实现
exec 分配栈帧 + 装入参数	Exec 七步，fakeStack+MemCopy 入用户栈
其余页由缺页中断分配	分支 A/B/C/D 全场景覆盖
释放时维护 Page[]+ 哈希表	Exit: FreeMemory(逐页)+ht.Release(逐文本页)
识别非法访问	FindVMArea→NULL→sigsegv
扩展堆栈	分支 B: 三重判定 +SStack 直写硬 PTE
代码/只读数据调页	分支 D: 哈希表 Lookup→ 共享只读
数据段首次访问	分支 C: 分配帧 + 零填充，VMA_DATA
堆首次访问	分支 C: 分配帧 + 零填充，VMA_HEAP
栈首次访问	分支 B: SStack 分配页 + 零填充
写时复制	分支 A: 三段式搬运，Page[old]-
定时复位	EvictOnePage 时钟算法: Accessed 复位 + 受害者
七大数据结构	全部实现（5.2 节）
交换区管理	SwapperManager+ForSystemUser bit0
进程终止	Exit 三阶段（5.8 节）

6 总结

本实验在 UNIX V6++ V1 上完成五项功能集成, 14 文件修改, vmtest 5/5 PASS, 15 项要求对标通过。

离散化: BitMap 替代 MapNode, $O(n) \rightarrow O(N/64)$, Page[] 为 COW 奠基。**滚动条:** 200 行缓冲 + 视口。**Shell:** 用户态行编辑 + Tab 补全 + 历史。**系统调用:** 49/50/51。**虚拟存储器:** 四分支缺页处理器 + DJB2 哈希表 + VMArea 链表 + 时钟置换 + COW 引用计数全周期 (含 Fork 计数漏计 Bug 修复)。